

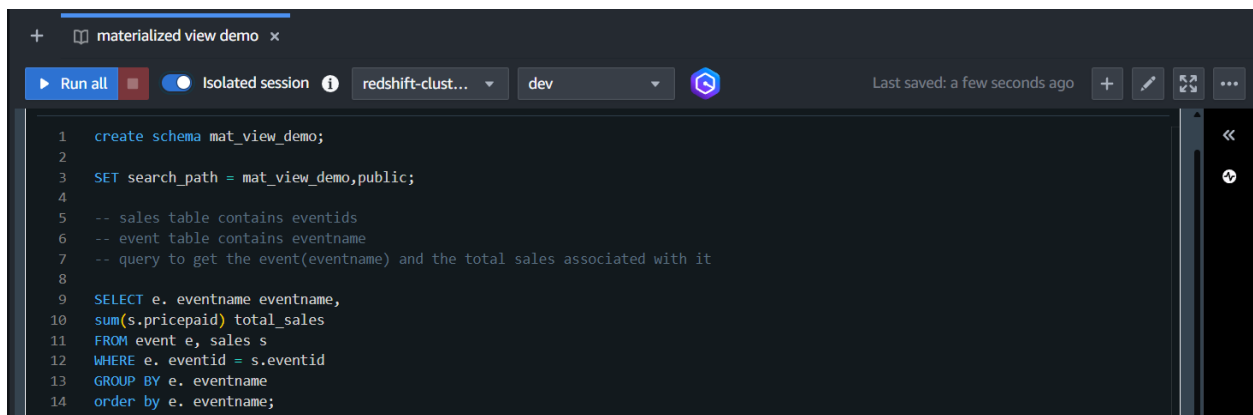
Materialized View in Amazon Redshift

To Begin with the Lab

Summary of the Lab

In this lab, a Materialized View named `tickets_mv` was created in **Amazon Redshift** using sales and event data from the Tikit sample database. A query joining the sales and event tables was executed first to analyze event-wise total sales. The query was then converted into a Materialized View to cache results and improve analytics performance for repetitive queries and dashboards. Base table data was later updated to demonstrate that Materialized Views use cached data until explicitly refreshed. Finally, the Materialized View was refreshed manually and validated successfully by confirming that the updated sales values were reflected in the query output.

- Navigate to **Amazon Redshift** → open **Redshift Query Editor v2**
- Create a schema for the lab data
- Connect to the Redshift cluster and select the dev database
- Create a new query editor tab for the lab

The screenshot shows the Amazon Redshift Query Editor v2 interface. At the top, there's a tab labeled 'materialized view demo'. Below the tab, there are buttons for 'Run all', 'Isolated session', and a dropdown menu showing 'redshift-clust...' and 'dev'. To the right, it says 'Last saved: a few seconds ago'. The main area contains a SQL query:

```
1 create schema mat_view_demo;
2
3 SET search_path = mat_view_demo,public;
4
5 -- sales table contains eventids
6 -- event table contains eventname
7 -- query to get the event(eventname) and the total sales associated with it
8
9 SELECT e. eventname eventname,
10 sum(s.pricepaid) total_sales
11 FROM event e, sales s
12 WHERE e. eventid = s.eventid
13 GROUP BY e. eventname
14 order by e. eventname;
```

CREATE SCHEMA `mat_view_demo`;

- Set the schema search path

SET `search_path` TO `mat_view_demo, public`;

- Run the base query to calculate event-wise total sales

The screenshot shows a Redshift SQL editor interface. The top bar includes a 'Run all' button, an 'Isolated session' toggle, and dropdown menus for 'redshift-clust...' and 'dev'. The main area contains the following SQL code:

```

8
9 SELECT e.eventname eventname,
10 sum(s.pricepaid) total_sales
11 FROM event e, sales s
12 WHERE e.eventid = s.eventid
13 GROUP BY e.eventname
14 order by e.eventname;
15
16 -- create materialized view
17 CREATE MATERIALIZED VIEW tickets_mv
18 AS (SELECT e.eventname eventname,
19 sum(s.pricepaid) total_sales
20 FROM event e, sales s
21 WHERE e.eventid = s.eventid
22 GROUP BY e.eventname);
23
24 -- check the view data
25 select * from mat_view_demo.tickets_mv order by eventname;
26
27 -- update the results
28 update sales s set pricepaid= pricepaid*0.5 where eventid in (select eventid from event where eventname = '.38 Special');

```

Below the code, the 'Result 1 (100)' tab is active, showing a table with 100 rows. The first two rows are visible:

eventname	total_sales
.38 Special	134386
3 Doors Down	125816

```

SELECT e.eventname eventname,
       SUM(s.pricepaid) total_sales
FROM event e, sales s
WHERE e.eventid = s.eventid
GROUP BY e.eventname
ORDER BY e.eventname;

```

- Verify that the query returns 100 rows
- Create the Materialized View using the same query

```

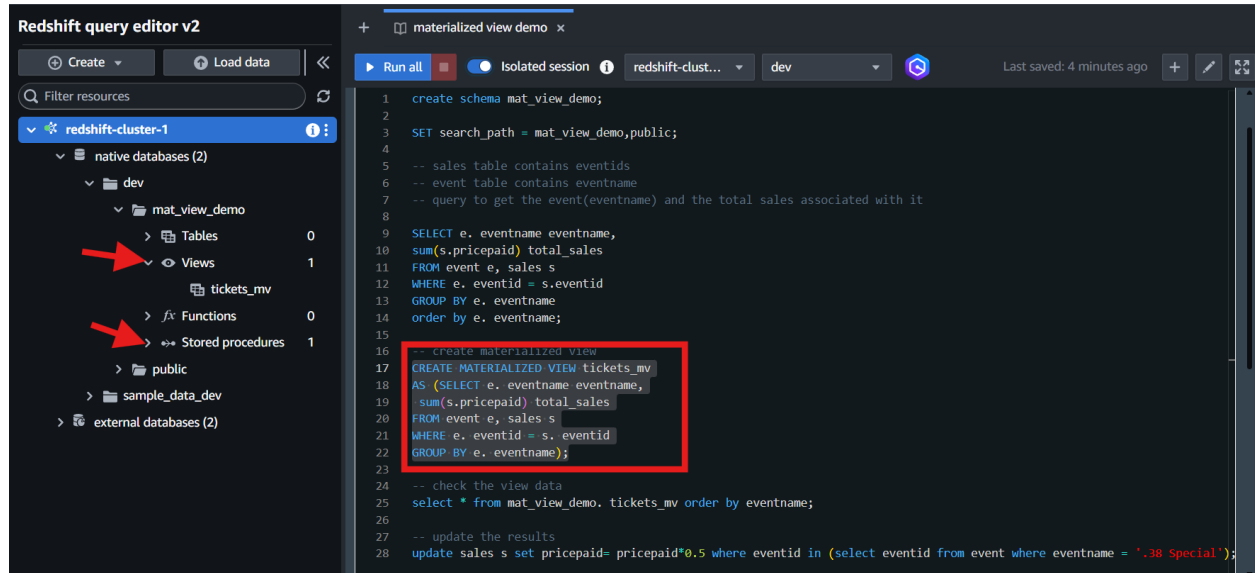
CREATE MATERIALIZED VIEW tickets_mv AS
SELECT e.eventname eventname,
       SUM(s.pricepaid) total_sales

```

FROM event e, sales s

WHERE e.eventid = s.eventid

GROUP BY e.eventname;

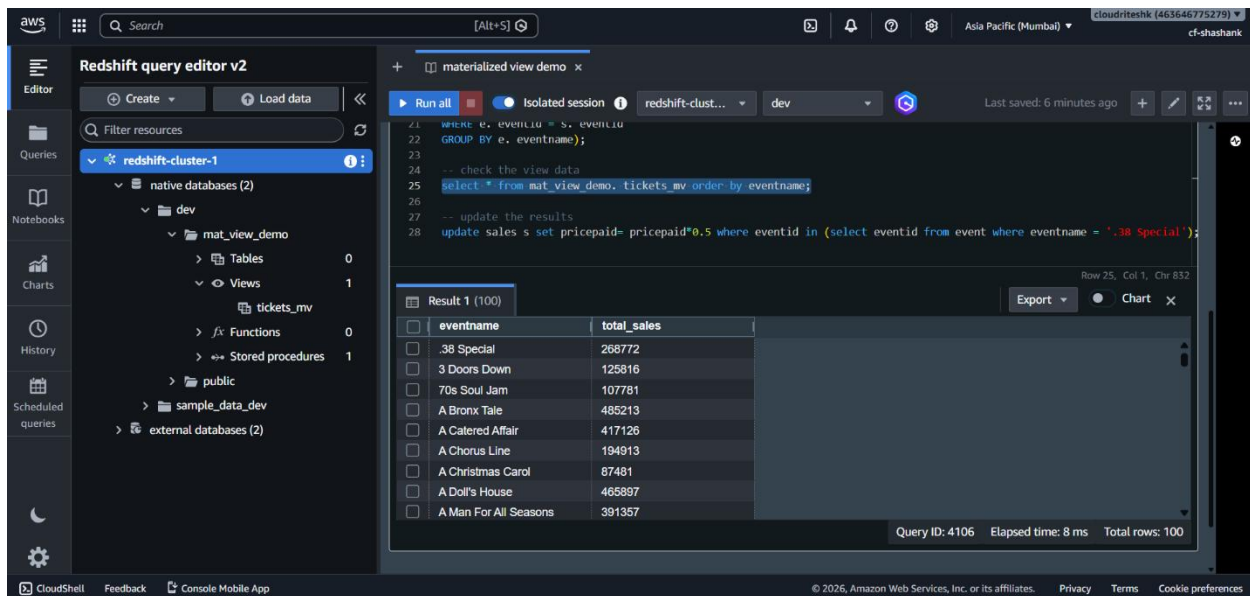


- Refresh the left navigation panel inside **Redshift Query Editor v2**
- Verify that the mat_view_demo schema now contains:
 - One View
 - One Internal Table
- Query the Materialized View

SELECT *

FROM mat_view_demo.tickets_mv

ORDER BY eventname;



- Verify that the Materialized View returns the same 100 rows as the original query
- Update the sales table data for the 0.38 Special event

UPDATE sales

SET pricepaid = pricepaid * 0.5

WHERE eventid IN (

SELECT eventid

FROM event

WHERE eventname = '0.38 Special');

The screenshot shows the Redshift query editor v2 interface. On the left, the 'native databases (2)' section is expanded, showing 'dev' and 'public' schemas. The 'dev' schema contains a materialized view 'mat_view_demo' with a table 'tickets_mv'. The main editor area shows the following SQL code:

```
22 GROUP BY e.eventname);
23
24 -- check the view data
25 select * from mat_view_demo.tickets_mv order by eventname;
26
27 -- update the results
28 update sales s set pricepaid= pricepaid*0.5 where eventid in (select eventid from event where eventname = '.38 Special');
29
30 -- refresh the view data
31 refresh materialized view mat_view_demo.tickets_mv;
```

The 'Result 1' summary is displayed below the query:

```
Summary
Affected rows: 426
Query ID: 4123
Elapsed time: 775ms
Result set query:
/* RDEV2-TyTuHvrvVA */
update sales s set pricepaid= pricepaid*0.5 where eventid in (select eventid from event where eventname = '.38 Special');
```

- Verify that approximately 426 rows were updated
- Query the Materialized View again

SELECT *

FROM mat_view_demo.tickets_mv

ORDER BY eventname;

The screenshot shows the Redshift query editor v2 interface. The SQL code in the editor is:

```
23
24 -- check the view data
25 select * from mat_view_demo.tickets_mv order by eventname;
26
27 -- update the results
28 update sales s set pricepaid= pricepaid*0.5 where eventid in (select eventid from event where eventname = '.38 Special');
29
30 -- refresh the view data
31 refresh materialized view mat_view_demo.tickets_mv;
```

The 'Result 1 (100)' is displayed below the query, showing a table with two columns: 'eventname' and 'total_sales'.

eventname	total_sales
.38 Special	268772
3 Doors Down	125816
70s Soul Jam	107781
A Bronx Tale	485213
A Catered Affair	417126
A Chorus Line	194913
A Christmas Carol	87481
A Doll's House	465897
A Man For All Seasons	391357

At the bottom of the editor, the status bar shows: Query ID: 4164 Elapsed time: 6 ms Total rows: 100.

- Observe that old cached values are still displayed

- Refresh the Materialized View manually

REFRESH MATERIALIZED VIEW mat_view_demo.tickets_mv;

The screenshot shows the Redshift query editor v2 interface. On the left, the 'native databases (2)' section is expanded, showing 'dev' and 'mat_view_demo'. Under 'mat_view_demo', 'tickets_mv' is listed under 'Views'. The main query editor shows a SQL script with the following lines highlighted in a red box:

```
-- refresh the view data
refresh materialized view mat_view_demo.tickets_mv;
```

Below the query editor, the 'Result 1' summary is displayed, also highlighted in a red box. It contains an 'Info' section with the message: 'Materialized view tickets_mv was incrementally updated successfully.' The summary also shows 'Returned rows: 0', 'Query ID: 4193', 'Elapsed time: 5.6s', and 'Result set query:'.

- Query the Materialized View after refresh

SELECT *

FROM mat_view_demo.tickets_mv

ORDER BY eventname;

The screenshot shows the Redshift query editor v2 interface. The main query editor shows a SQL script with the following lines highlighted in a red box:

```
-- check the view data
select * from mat_view_demo.tickets_mv order by eventname;
```

Below the query editor, the 'Result 1 (100)' table is displayed, also highlighted in a red box. The table has two columns: 'eventname' and 'total_sales'. The data is as follows:

eventname	total_sales
.38 Special	67193
3 Doors Down	125816
70s Soul Jam	107781
A Bronx Tale	485213
A Catered Affair	417126
A Chorus Line	194913
A Christmas Carol	87481
A Doll's House	465897
A Man For All Seasons	391357

The status bar at the bottom shows 'Query ID: 4206', 'Elapsed time: 166 ms', and 'Total rows: 100'.

- Verify that the 0.38 Special event now shows updated sales values of approximately 0.5+ million

- Confirm that the Materialized View refresh completed successfully and reflected the latest base table data